Preprint — not peer reviewed. Software: https://github.com/gkerherve/KherveSheet, DOI 10.5281/zenodo.23317839

KherveSheet: an Excel-like spreadsheet for scientists, with Python in every cell and AI assistants that work through the grid

Gwilherm Kerherve[1] (ORCID 0000-0002-6449-1828)

[1] Department of Materials, Imperial College London, London SW7 2AZ, United Kingdom

12 October 2026

Summary

KherveSheet is a desktop spreadsheet made for scientists. It looks and works like the spreadsheets researchers already use: a grid, formulas, cell formatting, charts and `.xlsx` import and export, with about 290 built-in functions. Behind that familiar interface it adds what experimental work needs:

- **Python in any cell.** A `=PY` cell holds a complete program, with imports, functions, loops and network access. It reads the grid with `ks("A1:B10")`, writes back with `ks_set(...)`, and has NumPy, SciPy, pandas and Matplotlib available. Its last expression becomes the cell's value: a number, a block of values, or an embedded Matplotlib figure. All `=PY` cells share one namespace, and cells can poll live data, for example instrument logs or web APIs.
- **Charts for publication.** Native charts have a full journal-style vocabulary: reversed axes, components filled down to a baseline, minor ticks with independent directions on each edge, annotations and export to PNG, SVG or PDF at any resolution. Where a native chart is not enough, a `=PY` cell can return any Matplotlib figure.
- **Analysis tools.** These include an Excel-style Solver, 23 trendline models with goodness-of-fit statistics, and lmfit-based peak fitting (Newville et al. 2014) over Shirley, Tougaard or polynomial backgrounds.
- **Ready-made scientific workbooks.** KherveSheet ships with more than 180 templates and worked examples, about 150 of them in science, engineering and data processing, and 59 written as `=PY` programs. They cover spectroscopy (XPS, Auger, UV-Vis, FTIR, Raman, EDX, XRF, XRD), materials characterisation (BET, TGA, DSC, Tauc plots, Hall effect), kinetics and thermodynamics, signal processing, statistics and Monte Carlo simulation. A set of graded exercises supports undergraduate teaching from first to fourth year.
- **AI assistants that work in the grid.** A built-in Model Context Protocol (MCP) server (Anthropic 2024) lets any local MCP client, such as Claude Desktop, Claude Code, Cursor, VS Code or LM Studio, read and edit the open workbook through 36 documented tools. An assistant does its analysis by writing `=PY` cells, so every number it produces traces back to code a person can read, edit and re-run.

KherveSheet is written in Python with PyQt5 and released under GPL-3.0 for Windows and macOS. Its calculation engine has no Qt dependency and also runs in the browser through Pyodide.

Statement of need

Spreadsheets remain the tool most scientists reach for first: to look at an instrument export, tabulate results or make a quick plot. General-purpose spreadsheets serve this work poorly. Their numerical tools are limited, their charts need extensive manual work to reach journal conventions, and anything beyond a formula means leaving for a programming environment. Their failure modes are also well documented. Automatic type conversion turned gene symbols into dates in roughly a fifth of the genomics papers that came with Excel gene lists (Ziemann et al. 2016). A range error in a widely cited macroeconomics spreadsheet went

unnoticed until others tried to reproduce it (Herndon et al. 2014). In both cases the logic was hidden in formulas and manual steps.

Notebooks such as Jupyter (Kluyver et al. 2016) solve the transparency problem, but they ask experimental scientists to give up the grid they think in. KherveSheet keeps the grid and puts real code in it. Data stays visible in cells, the analysis is readable Python next to it, and a template library means a user starts from a working XPS quantification or BET analysis instead of a blank sheet.

The same design addresses a newer problem. Large language model (LLM) assistants can process a dataset in seconds, but their intermediate steps usually stay inside a conversation, which makes the result hard to check or reproduce. In fields such as X-ray photoelectron spectroscopy (XPS), inconsistent peak fitting is already a recognised source of error in the literature (Major et al. 2020), so an opaque assistant is a risk. In KherveSheet the assistant has no private workspace. The grid is the shared interface, and everything the assistant computes is a cell the scientist can audit. Every remote change is one undo step. An access level (read only, edit or full) decides whether a client may run Python at all, and the MCP server listens only on the local machine, so unpublished data never leaves the user's computer.

The intended users are experimental scientists, such as surface scientists, chemists, materials scientists and physicists, together with their students.

State of the field

Python inside a spreadsheet has been explored before. Pyspread (Manns 2024) makes every cell a Python expression. Microsoft's Python in Excel (Microsoft 2023) adds a `=PY` function that runs in Microsoft's cloud. Reactive notebooks such as marimo (Agrawal and Scolnick 2024) offer Python with live outputs, but they are not spreadsheets. KherveSheet differs in four ways. Its cells hold full multi-line programs that run locally. Its interface, tools and template library are aimed at laboratory science rather than general office work. Its native charts follow scientific plotting conventions. And it is designed for AI assistants that have to work through visible cells, a model that MCP has only recently made possible.

Example: fitting the Ti 2p spectrum of TiO_2 through MCP

As a worked example, we fit the Ti 2p core-level spectrum of a TiO_2 reference sample. The spectrum was recorded with monochromatic Al Kα radiation. An MCP client placed five `=PY` cells in an empty workbook. Apart from formatting and saving, the client did nothing except call `set_python_cell`. Every step of the analysis is code in the grid (Figure 1):

- **A1** loads the instrument export with pandas and writes binding energy and counts into columns A and B.
- **C1** defines an iterative Shirley background (Shirley 1972) and uses it to seed the fit.
- **D1** fits the spectrum over the full 448–475 eV range with SciPy (Virtanen et al. 2020). The model has a Ti^{4+} $2p_{3/2}$/$2p_{1/2}$ Voigt doublet in which the $2p_{1/2}$ line has its own widths, to allow for Coster–Kronig broadening. It adds a Gaussian for the Ti^{4+} charge-transfer satellite, and an *active* Shirley background (Herrera-Gomez et al. 2014) that is built from the fitted peaks and refined with them. The components, envelope and residual are written into columns D–H.
- **I1** writes a results table with positions, FWHM (Olivero–Longbothum approximation (Olivero and Longbothum 1977)), heights, areas, the spin–orbit splitting with its uncertainty, and the reduced χ^2.
- **T1** runs a Monte Carlo error analysis. It draws 300 synthetic spectra from the fitted model with Poisson counting noise, refits each one, and writes every refit (T12 onwards) and a summary table (T3:X9) into the grid.
- **I40** returns a Matplotlib figure with normalised residuals, embedded in the grid as a quick visual check of the fit.

The publication figure (Figure 2) is a native KherveSheet chart, not Python code. The client built it with `insert_chart` over columns A–F and filled each component down to the Shirley background with `style_chart`.

It drew the spin–orbit splitting and the satellite offset with `annotate_chart`. With `configure_axis` it reversed the binding-energy axis, following XPS conventions (Major et al. 2020), and set a full frame with minor ticks. It then exported the chart with `export_chart` at 300 dpi. The chart stays linked to the cells, so re-running a =PY cell redraws it.

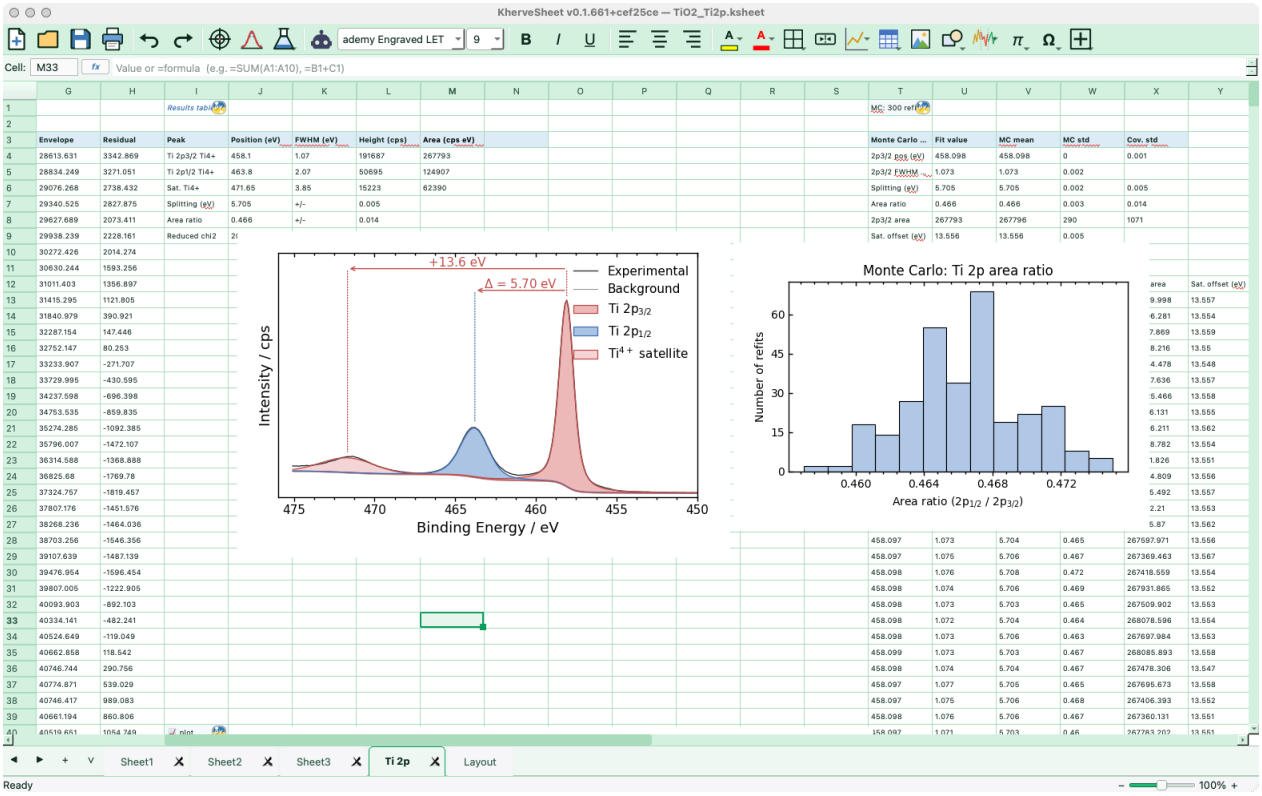


Figure 1: KherveSheet after the MCP session. Every number comes from =PY cells in row 1: the envelope and residual (columns G–H), the results table (I3:M9), and the Monte Carlo summary (T3:X9) with its 300 refits below it. The Ti 2p spectrum and the Monte Carlo histogram are native KherveSheet charts over those cells.

The fit gives the Ti $2p_{3/2}$ line at 458.10 eV with an FWHM of 1.07 eV, a spin–orbit splitting of 5.705 ± 0.005 eV, and a satellite 13.6 eV above the main line. These values agree with the established Ti^{4+} signature of TiO_2 (Biesinger et al. 2010). As a cross-check, we compared the same spectrum fitted independently in KherveFitting. Both programs give the same $2p_{3/2}$ FWHM (1.07 eV) and splitting (5.70 eV), and their $2p_{3/2}$ areas differ by 1.3%. The 0.14 eV difference in position comes from the charge correction applied in the KherveFitting file and is not a difference between the fits.

Monte Carlo uncertainties

Standard errors from the covariance matrix assume the model is exact. With a reduced χ^2 of 20 here, that assumption clearly fails. The =PY cell in T1 therefore estimates the counting-statistics contribution directly: it generates 300 Poisson realisations of the fitted spectrum and refits each one from the same starting point (Table 1). The client then built a native histogram of the 300 area ratios with one `insert_chart` call (Figure 3).

Table 1: Monte Carlo (N = 300, Poisson noise) versus covariance standard deviations.

Parameter	Fit value	MC std	Covariance std
Ti $2p_{3/2}$ position (eV)	458.098	< 0.001	0.001
Ti $2p_{3/2}$ FWHM (eV)	1.073	0.002	—
Spin–orbit splitting (eV)	5.705	0.002	0.005
Area ratio $2p_{1/2}/2p_{3/2}$	0.466	0.003	0.014
Ti $2p_{3/2}$ area (cps eV)	267 793	290	1 071
Satellite offset (eV)	13.556	0.005	—

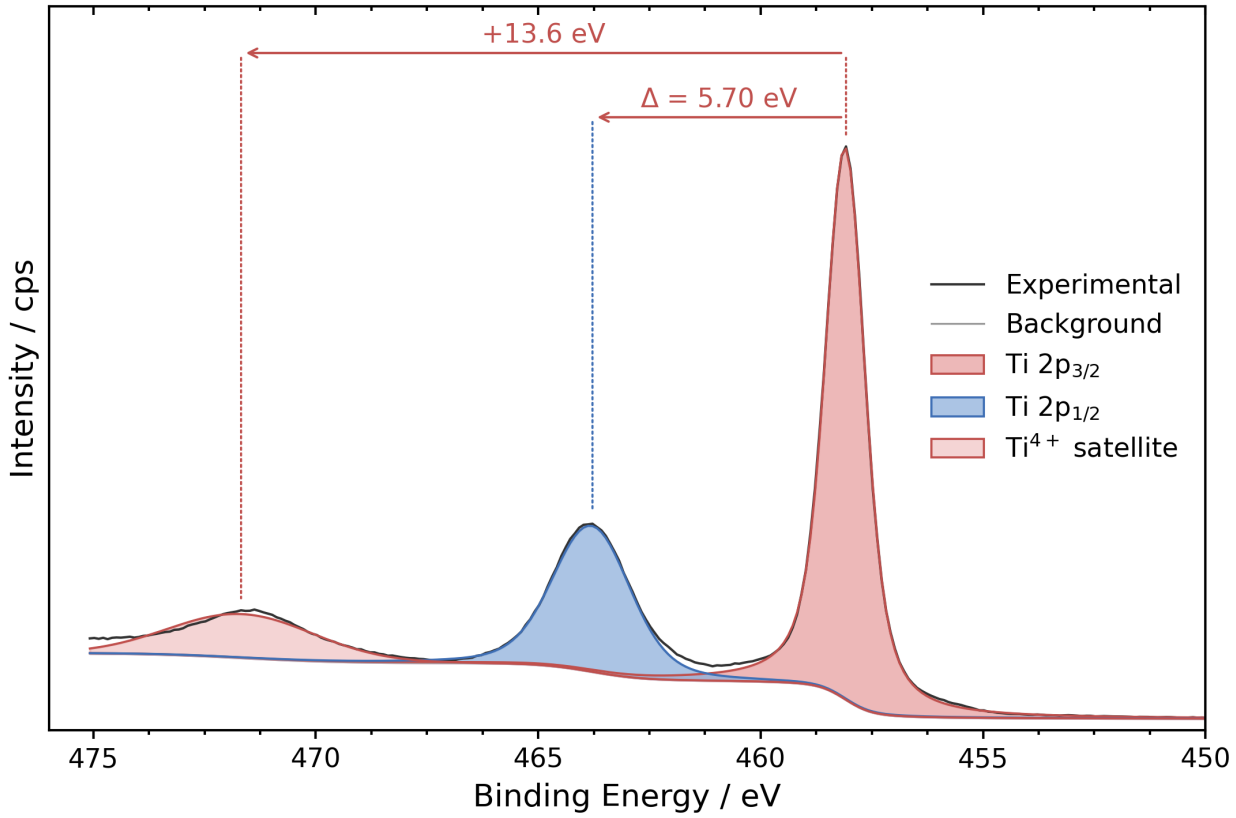


Figure 2: Ti 2p spectrum of TiO_2 as a native KherveSheet chart built over MCP. The fit was computed in =PY cells: a Voigt spin–orbit doublet (red, blue) and the Ti^{4+} charge-transfer satellite over an active Shirley background. Arrows mark the spin–orbit splitting and the satellite offset.

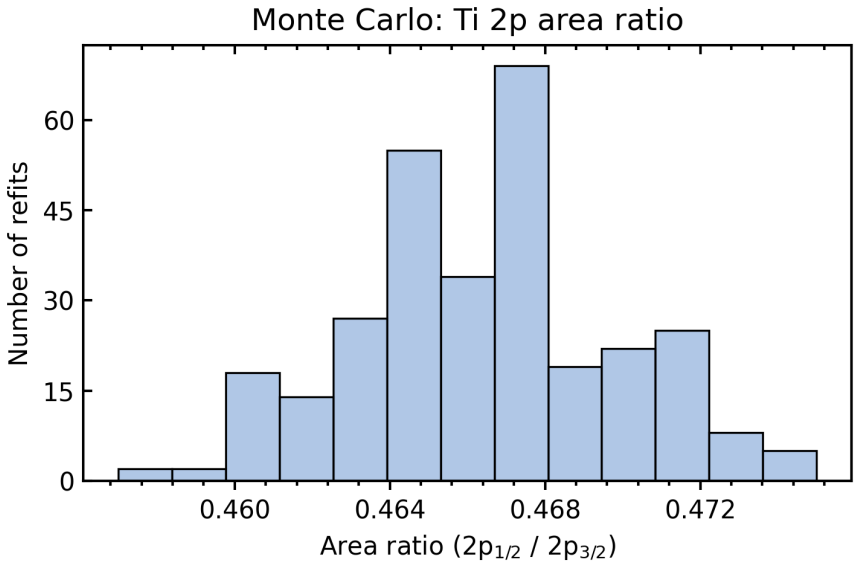


Figure 3: Distribution of the Ti 2p area ratio over 300 Monte Carlo refits, as a native KherveSheet histogram over the T-column refits.

The Monte Carlo spread is three to five times smaller than the covariance estimate. On data with this many counts, Poisson noise is therefore not the limiting factor. The uncertainty in quantities such as the area ratio comes mainly from the model: background, line shape and the satellite. The covariance errors, which are scaled by the reduced χ^2, partly reflect that misfit, and the Monte Carlo errors do not. Because both analyses sit side by side in the grid as readable code, a reader can see this distinction directly and test alternative models by editing a single cell.

The script that places these cells, the full log of MCP calls (`mcp_calls.jsonl`) and the resulting workbook are in the repository under `paper/demo/`. Re-running the script rebuilds the workbook and the figure.

AI usage disclosure

Generative AI was used in this work. Claude (Anthropic; Claude Opus and Sonnet models, through Claude Code, 2025–2026) assisted with writing parts of the KherveSheet source code, tests and documentation, and with drafting and editing this manuscript. In the worked example, Claude also drove the MCP session that placed the `=PY` cells. The author designed the software and its architecture, made all core design decisions, chose the scientific analysis and models, and reviewed, tested and validated all AI-assisted code, results and text, and takes full responsibility for them.

Acknowledgements

We thank the users of the Kherve tools at Imperial College London for feedback.

References

Agrawal, Akshay and Myles Scolnick (2024). *marimo: a reactive notebook for Python*. URL: https://github.com/marimo-team/marimo.

Anthropic (2024). *Model Context Protocol specification*. URL: https://modelcontextprotocol.io.

Biesinger, Mark C. et al. (2010). "Resolving surface chemical states in XPS analysis of first row transition metals, oxides and hydroxides: Sc, Ti, V, Cu and Zn". In: *Applied Surface Science* 257.3, pp. 887–898. DOI: 10.1016/j.apsusc.2010.07.086.

Herndon, Thomas, Michael Ash, and Robert Pollin (2014). "Does high public debt consistently stifle economic growth? A critique of Reinhart and Rogoff". In: *Cambridge Journal of Economics* 38.2, pp. 257–279. DOI: 10.1093/cje/bet075.

Herrera-Gomez, A. et al. (2014). "Practical methods for background subtraction in photoemission spectra". In: *Surface and Interface Analysis* 46.10–11, pp. 897–905. DOI: 10.1002/sia.5453.

Kluyver, Thomas, Benjamin Ragan-Kelley, Fernando Pérez, et al. (2016). "Jupyter Notebooks – a publishing format for reproducible computational workflows". In: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pp. 87–90. DOI: 10.3233/978-1-61499-649-1-87.

Major, George H. et al. (2020). "Practical guide for curve fitting in X-ray photoelectron spectroscopy". In: *Journal of Vacuum Science & Technology A* 38.6, p. 061203. DOI: 10.1116/6.0000377.

Manns, Martin (2024). *pyspread: a non-traditional spreadsheet in Python*. URL: https://pyspread.gitlab.io.

Microsoft (2023). *Python in Excel*. URL: https://support.microsoft.com/office/introduction-to-python-in-excel-55643c2e-ff56-4168-b1ce-9428c8308545.

Newville, Matthew et al. (2014). *LMFIT: Non-Linear Least-Square Minimization and Curve-Fitting for Python*. DOI: 10.5281/zenodo.11813.

Olivero, J. J. and R. L. Longbothum (1977). "Empirical fits to the Voigt line width: a brief review". In: *Journal of Quantitative Spectroscopy and Radiative Transfer* 17.2, pp. 233–236. DOI: 10.1016/0022-4073(77)90161-3.

Shirley, D. A. (1972). "High-resolution X-ray photoemission spectrum of the valence bands of gold". In: *Physical Review B* 5.12, pp. 4709–4714. DOI: 10.1103/PhysRevB.5.4709.

Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature Methods* 17, pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

Ziemann, Mark, Yotam Eren, and Assam El-Osta (2016). "Gene name errors are widespread in the scientific literature". In: *Genome Biology* 17, p. 177. DOI: 10.1186/s13059-016-1044-7.